

Extended thinking in manual mode gives you direct control over how much Claude thinks. You set a thinking token budget on each request with `thinking: {type: "enabled", budget_tokens: N}`, and Claude thinks against that budget before it starts its final answer. Manual mode remains useful when your workload requires predictable latency or precise control over thinking costs. This page covers how to set and tune the budget, how manual mode interacts with interleaved thinking and prompt caching, and how to migrate to adaptive thinking.

To learn how thinking itself works, including thinking blocks and the response shape, the `display` parameter, streaming, thinking with tool use, and encryption, see the [thinking overview](#).

Supported models

Extended thinking availability per model, including the models where extended thinking is the only mode, is listed in the [per-model configuration table](#).

How to use extended thinking

Here is an example of using extended thinking in the Messages API:

Python

```
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[
        {
            "role": "user",
            "content": "Are there an infinite number of prime numbers such that
n mod 4 == 3?",
        }
    ],
)

# The response contains summarized thinking blocks and text blocks
for block in response.content:
    match block.type:
        case "thinking":
            print(f"\nThinking summary: {block.thinking}")
        case "text":
            print(f"\nResponse: {block.text}")
```

To turn on manual extended thinking, add a `thinking` object with `type` set to `enabled` and a `budget_tokens` value.

The `budget_tokens` parameter sets a target for how many tokens Claude can use for its internal reasoning process. Larger budgets can improve response quality by enabling more thorough analysis for complex problems.

Budget rules and tuning

`budget_tokens` must satisfy these constraints:

- **Minimum of 1,024 tokens.** The API rejects smaller values.
- **Less than `max_tokens`.** Thinking tokens count toward the `max_tokens` limit for the turn, so the budget must leave room for the final response. The one exception is [interleaved thinking](#), where `budget_tokens` can exceed `max_tokens` because the budget spans all thinking blocks within one assistant turn.
- **No cache pre-warming.** Because `budget_tokens` must be less than `max_tokens`, extended thinking cannot be combined with `max_tokens: 0` ([cache pre-warming](#)).

The budget is a target rather than a strict cap. Actual token usage varies with the task, and Claude may stop reasoning well before the budget is exhausted; `max_tokens` remains the hard ceiling on total output.

On Claude Opus 4.5, the only extended-thinking-only model that supports [effort](#), `effort` shapes the overall response while `budget_tokens` sets thinking depth; set both.

To tune the budget:

- Match the starting point to the task. For simple tasks, start near the 1,024-token minimum and increase incrementally to find the optimal range for your use case. For complex tasks, start with a larger budget of 16,000 tokens or more and adjust to your latency and quality needs. Higher budgets enable more comprehensive reasoning, with diminishing returns that depend on the task, and at the cost of increased latency. For critical tasks, test different settings to find the right balance.
- For thinking budgets above 32k, use [batch processing](#) to avoid networking issues. Pushing the model to think beyond 32k tokens produces long-running requests that can hit system timeouts and open-connection limits.

To track what a budget actually costs you, monitor the `usage.output_tokens_details.thinking_tokens` field in the response, which reports how many of the billed output tokens were internal reasoning. When streaming, this breakdown appears only on the final `message_delta` event.

When you are ready to move off manual budgets, see [Migrating to adaptive thinking](#).

Interleaved thinking in manual mode

Interleaved thinking lets Claude think between tool calls within a single assistant turn, reasoning about each tool result before determining what to do next. For the concept, the turn structure, and how it behaves on adaptive-thinking models, see [interleaved thinking](#) in the thinking overview. This section covers how to enable it when you use manual `type: "enabled"` thinking.

On Claude Opus 4.5, Claude Sonnet 4.5 (deprecated), and earlier Claude 4 models, add the `interleaved-thinking-2025-05-14` [beta header](#) to your API request.

The 4.6 generation splits in manual mode:

- **Claude Sonnet 4.6:** the beta header with manual `type: "enabled"` is still functional but deprecated. Prefer [adaptive thinking](#), which interleaves automatically with no header.
- **Claude Opus 4.6:** manual mode has no interleaved thinking at all. Only its adaptive mode interleaves, so switch to `thinking: {type: "adaptive"}` if you need reasoning between tool calls on this model.

Claude Haiku 4.5 does not support interleaved thinking. On the Claude API, the beta header is accepted but ignored.

Two more considerations for interleaved thinking in manual mode:

- `budget_tokens` can exceed `max_tokens` here; the [budget rules](#) explain this exception.
- Interleaved thinking is only supported for [tools used through the Messages API](#).

How platforms treat the beta header differs. The Claude API and [Claude Platform on AWS](#) accept `interleaved-thinking-2025-05-14` on any model and ignore it where unsupported. Acceptance is not the same as effect: on models that reject `type: "enabled"` (4.7 and later) or lack manual-mode interleaving (Claude Opus 4.6), the header has no manual-mode effect; adaptive thinking interleaves automatically there.

Partner-operated platforms ([Amazon Bedrock](#) and [Google Cloud](#)) likewise accept the header on any model without returning an error, and ignore it on models that don't support interleaved thinking.

Turn structure in manual mode

The general turn-structure rules, including the single-turn tool-use loop, mid-turn conflict handling, and toggling thinking between turns, are on [Thinking with tool use](#).

Manual mode adds one requirement: the final assistant turn of a thinking-enabled request must begin with a thinking block ([adaptive thinking](#) drops that requirement). Changing the thinking configuration between turns also invalidates prompt caching; see the following section.

Prompt caching in manual mode

Manual mode adds one rule on top of the mode-neutral caching behavior described in [thinking and prompt caching](#): changing `budget_tokens` between requests invalidates cache breakpoints, just as switching thinking modes does, because the budget value is rendered into the prompt. Message-level breakpoints always miss after a budget change; whether tool and system-prompt breakpoints miss too depends on where the model renders the configuration.

In practice, pick a budget and hold it stable for the life of a cached conversation. Running a multi-turn conversation with message-level caching on Claude Sonnet 4.6 and changing the budget on the third request from 4,000 to 8,000 tokens shows the invalidation directly:

Output

```
First request - establishing cache
First response usage: { cache_creation_input_tokens: 1370,
cache_read_input_tokens: 0, input_tokens: 17, output_tokens: 700 }

Second request - same thinking parameters (cache hit expected)
Second response usage: { cache_creation_input_tokens: 0,
cache_read_input_tokens: 1370, input_tokens: 303, output_tokens: 874 }

Third request - different thinking budget (cache miss expected)
Third response usage: { cache_creation_input_tokens: 1370,
cache_read_input_tokens: 0, input_tokens: 747, output_tokens: 619 }
```

The third request re-creates the cache (`cache_creation_input_tokens=1370`, `cache_read_input_tokens=0`) because the budget changed between requests. For a runnable version of the same experiment in adaptive mode, where the effort level plays the cache role that `budget_tokens` plays here, see [Prompt caching](#) on the steering page.

Shared mechanics

Most thinking behavior is mode neutral and documented once on the [Thinking](#) page. Everything there applies in manual mode too:

- [Controlling thinking display](#)
- [Streaming thinking](#)
- [Thinking with tool use](#), including [preserving thinking blocks](#)
- [Thinking and prompt caching](#)
- [Thinking and the context window](#)
- [Thinking encryption](#)
- [Pricing](#) (on the [Steering thinking](#) page)

Migrating to adaptive thinking

If your model supports only extended thinking (Claude Sonnet 4.5 (deprecated), Claude Opus 4.5, Claude Haiku 4.5, and earlier Claude 4 models), no action is needed now: adaptive thinking is not available there, and `type: "adaptive"` [returns a 400 error](#). Keep `budget_tokens` until you move to a model that supports adaptive thinking, then apply the mapping that follows.

You need to migrate off `type: "enabled"` if:

- You use Claude Opus 4.6 or Claude Sonnet 4.6, where `budget_tokens` is deprecated.
- You use Claude 4.7 or a later model, such as Claude Opus 5.5, Claude Sonnet 5, Claude Sonnet 5.5, Claude Fable 5.1, or Claude Haiku 5.5, where `type: "enabled"` returns a 400 error.

The mapping is small: remove `budget_tokens`, set `thinking: {type: "adaptive"}`, and control reasoning depth with `output_config: {effort: ...}` instead of a token budget.

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 16000,
  "thinking": {
    "type": "enabled",
    "budget_tokens": 10000
  }
}
```

becomes:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 16000,
  "thinking": {
    "type": "adaptive"
  },
  "output_config": {
    "effort": "high"
  }
}
```

`effort: "high"` matches the API default; it appears here only to show where the depth control now lives, and omitting it produces identical behavior.

Expect a behavioral difference, not just a syntax change. With a fixed budget, Claude thinks on every request. With adaptive thinking, Claude determines whether and how much to think on each request, and at lower [effort](#) settings it may skip thinking entirely on easy inputs. You can also remove the `interleaved-thinking-2025-05-14` beta header after migrating:

adaptive thinking interleaves automatically, and the Claude API ignores the header on these models. Thinking block preservation changes too: Claude Opus 4.5 and models numbered 4.6 and higher keep prior turns' thinking blocks in context and bill them as input, where Claude Sonnet 4.5 (deprecated), Claude Haiku 4.5, and earlier models stripped them; see [thinking block preservation by model](#).

Switching modes is a thinking-configuration change, so the first request after the switch invalidates cache breakpoints, as described in [Prompt caching in manual mode](#).

For full guidance, see [adaptive thinking](#), [effort](#), and the [model migration guide](#).

Next steps



Thinking

Learn how thinking works: blocks, display, streaming, and tool use.



Steering thinking

Let Claude determine when and how much to think on each request.



Thinking in tool and multi-turn workflows

Preserve thinking blocks and manage thinking across tool calls and turns.